Team MCP Gateway

WooCommerce store + our own MCP server + a team MCP gateway + chat agent,
all on Kubernetes with Helm

The full guide: **what** we built, **how** we built it, and **why**

Urumi AI FDE Internship · Round 1 take-home
Repo: github.com/dhanoliya-ji/team-mcp-gateway

Contents

1. The task in simple words
2. Words you need to know
3. The big picture
4. Tools and tech we used, and why
5. The order we built things in
6. Part 1: our WooCommerce MCP server
7. Part 2: the two extra MCP servers
8. Part 3: the gateway (file by file)
9. Part 4: the chat agent and its tool loop
10. Part 5: the React dashboard
11. Part 6: Kubernetes and Helm (template by template)
12. How one tool call travels, step by step
13. Security: keys, secrets, isolation
14. Problems we hit and how we fixed them
15. Testing: what we checked and what we saw
16. How to run it: local, production, upgrade, rollback
17. Demo video plan
18. Interview questions and simple answers
19. Limits and next steps

1. The task in simple words

Urumi asked us to build a small platform with **four layers**:

1. **A store.** One online shop (WordPress + WooCommerce) running on Kubernetes.
2. **Our own WooCommerce MCP server.** A program we write ourselves that gives AI tools to read and change the shop's products and orders.
3. **A gateway and a dashboard.** The admin adds MCP servers once. The gateway connects to all of them and shows all their tools through **one** address. Each team member uses their own API key.
4. **A chat agent.** A chat box in the dashboard. An AI model answers questions by calling the same tools through the gateway.

Rules: everything runs in Kubernetes, installed with **Helm**. The same chart must work on a laptop (Kind) and on a server (k3s). No passwords in the code. Every tool call goes into an audit log.

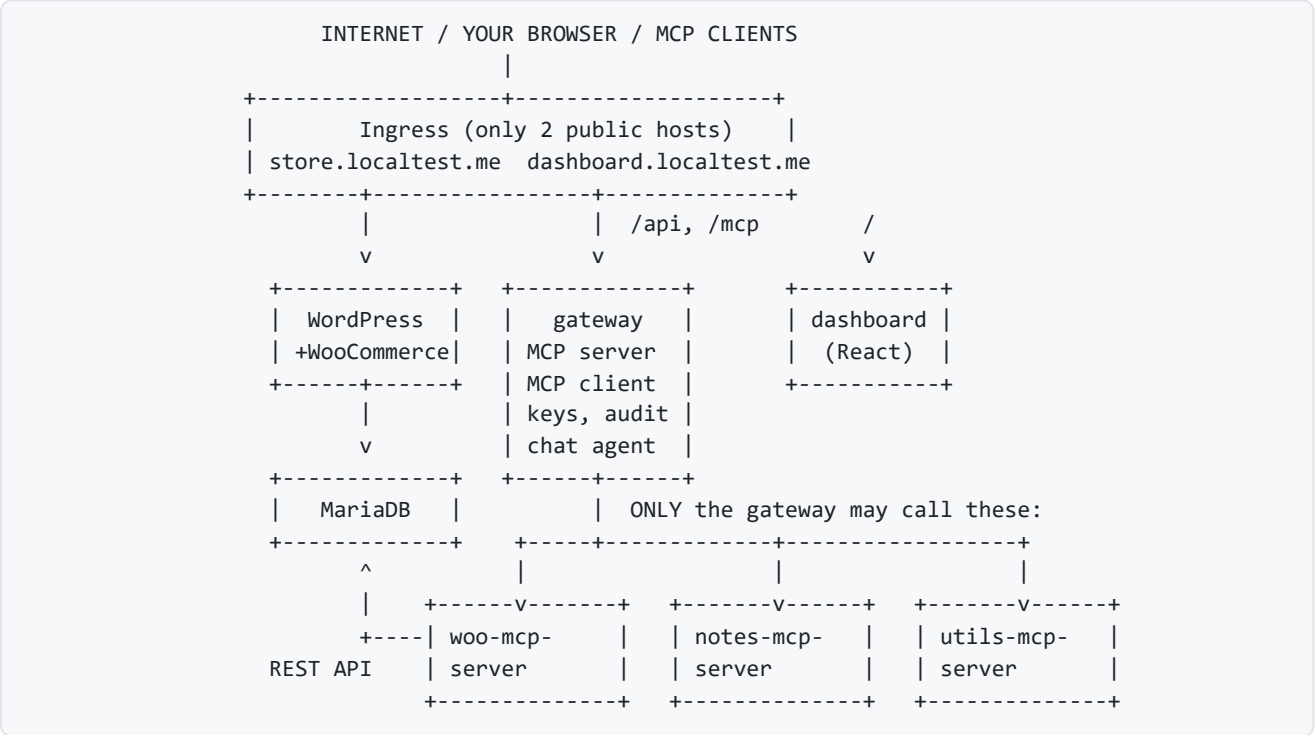
> **The short version:** one door (the gateway) in front of many MCP servers. Each person has their own key to the door. The door keeps the real keys of the servers hidden, writes down every visit, and can lock out a person or a server at any time.

2. Words you need to know

Word	Simple meaning
Kubernetes (K8s)	A system that runs many programs (in containers) on one or more computers, restarts them when they crash, and connects them with a network.
Container / image	An image is a packed program with everything it needs. A container is a running copy of an image.
Pod	The smallest thing Kubernetes runs. Usually one container.
Deployment	"Keep N copies of this pod running." If a pod dies, a new one starts.
StatefulSet	Like a Deployment, but each pod keeps a fixed name and its own disk. Good for databases.
Service	A fixed name and address for a group of pods, for example `wordpress`.
Ingress	The front door from the internet into the cluster. It sends `store.localtest.me` to WordPress, and so on.
PersistentVolumeClaim (PVC)	A request for a disk that survives pod restarts.
Secret	A Kubernetes object that holds passwords and keys.
NetworkPolicy	Firewall rules between pods: who may talk to whom.
Probe (readiness / liveness)	Kubernetes asks the pod "are you ready?" and "are you alive?". If not alive, it restarts it. If not ready, it sends no traffic.
Requests / limits	How much CPU and memory a pod is promised (request) and the most it may use (limit).
Helm / chart / values	Helm installs apps on Kubernetes. A chart is a folder of templates. Values fill in the templates (for example the domain name).
Kind	"Kubernetes in Docker": a real cluster on your laptop.
k3s	A small, real Kubernetes for servers (VPS).
MCP	Model Context Protocol: a standard way for AI apps to find and call tools. Messages are JSON-RPC.
MCP server / client	The server offers tools. The client (Claude Code, Cursor, our gateway) lists and calls them.
Streamable HTTP	The MCP transport we use: the client sends an HTTP POST to `/mcp` with a JSON-RPC message and gets the answer back.
Tool	One action an AI can do, with a name, a description and an input schema. For example `list_orders`.
Upstream server	A real MCP server behind the gateway (WooCommerce, notes, utils).

Hash	A one-way fingerprint of a text. You can check a key against it, but you cannot get the key back from it.
Encryption	Two-way: with the secret key you can get the original text back.
Audit log	A list of every tool call: who, what, when, how long, and if it worked.
LLM	Large Language Model, the AI that reads the chat and decides which tools to call.

3. The big picture



Who talks to whom:

- Shoppers use the store through the Ingress.
- Admins and members use the dashboard through the Ingress. The dashboard calls the gateway's `/api`.
- External MCP clients (Claude Code, Cursor, MCP Inspector) call the gateway's `/mcp` with their API key.
- The chat agent lives inside the gateway and calls the gateway's own `/mcp`, as the logged-in user.
- Only the gateway calls the three MCP servers. Only the WooCommerce MCP server calls WordPress's REST API.

4. Tools and tech we used, and why

Thing	What we used	Why
Language	JavaScript on Node.js 24	One language for all servers and the dashboard. Very simple code. Node 24 has SQLite built in.
MCP	Official MCP TypeScript SDK `@modelcontextprotocol/sdk` 1.32	It handles the protocol (initialize, tools/list, tools/call, Streamable HTTP) for us, both server and client side.
Web server	Express 5	Small and well known.
Input checks	zod	Describes each tool's inputs. The SDK turns it into JSON Schema for the AI.
Gateway database	SQLite (`node:sqlite`) on a volume	Small data (users, keys, servers, audit). No extra database pod needed.
Dashboard	React 19 + Vite, served by `nginx-unprivileged`	Asked for React. Vite builds fast. nginx-unprivileged runs as non-root.
Store	`wordpress:7.0-php8.3-apache` , `mariadb:11.4` , WooCommerce 11.2	Official images. WooCommerce 11 needs WordPress 7.
Store setup	WP-CLI (`wordpress:cli` image) in a Helm hook Job	Installs everything with no clicks, and is safe to run again.
Local Kubernetes	Kind v0.33 + ingress-nginx	A real cluster in Docker. Port 80/443 mapped to the laptop.
Packaging	One Helm chart, values-local.yaml / values-prod.yaml	Required. Same chart, only values change.
LLM	Any OpenAI-style API. Default: Groq, `llama-3.3-70b-versatile`	Free tier, fast, supports tool calling. Gemini or a local Ollama also work by changing two values.
Production images	GitHub Actions -> ghcr.io	The VPS pulls ready images.

5. The order we built things in

We followed the advice "a working core is worth more than an unfinished bonus". We built from the bottom up and tested each layer before the next. Every small step is its own Git commit.

1. **Repo start:** README, .gitignore, .gitattributes (Linux line endings, so shell scripts work in containers).

2. **WooCommerce MCP server:** REST helper -> tools -> HTTP server -> Dockerfile. Tested `tools/list` and `ping` locally with curl.
3. **Notes and utils MCP servers:** same pattern.
4. **Kind cluster** with ingress-nginx (in the background while coding).
5. **Gateway:** config -> database -> security helpers -> upstream client -> auth -> audit -> MCP endpoint -> SSRF check -> routes (login, servers, users, keys, audit) -> chat agent -> seed -> main file. Tested locally with two upstreams and one dead one.
6. **Dashboard:** project -> API helper -> app shell with login -> styles -> pages (servers, keys, members, audit, chat) -> Dockerfile.
7. **Helm chart:** Chart.yaml, values, helpers, Secret, MariaDB, WordPress, setup Job, 3 MCP servers, gateway, dashboard, Ingress, NetworkPolicies, NOTES, values files.
8. **First install,** fix problems (section 14), full test with a script.
9. **Stand-out:** tool permissions; hardening; docs; GitHub Actions; upgrade/rollback test.

6. Part 1: our WooCommerce MCP server

Folder: `woo-mcp-server/`. We wrote it ourselves (no existing WooCommerce MCP plugin).

6.1 How it talks to the store, and why

It uses the **WooCommerce REST API** (`/wc/v3/products` , `/wc/v3/orders`) over plain HTTP inside the cluster, with a **consumer key and secret** (Basic auth).

- **Why REST and not WP-CLI?** REST is the official API for products and orders, works over the network, and lets the MCP server be its own small pod with its own limits and firewall rules. WP-CLI would need to run inside the WordPress pod.
- **Why `?rest_route=` ?** It works even if WordPress "pretty links" are not set up.
- **Why the `X-Forwarded-Proto: https` header?** WooCommerce accepts key + secret only on HTTPS. Inside the cluster we use plain HTTP. The WordPress image trusts this header and acts as if the request was HTTPS. This is safe because WordPress is only reachable from the ingress controller and the woo-mcp-server pod (NetworkPolicy).

6.2 Files

File	What it does
`src/woo-api.js`	One function `wooRequest(method, path, query, body)` . Builds the URL, adds Basic auth and the header, 15 s timeout, throws a clear error if the store says no.
`src/tools.js`	Registers the tools with `server.registerTool(name, {description, inputSchema}, handler)` . Each handler calls `wooRequest` and returns a short JSON text.
`src/index.js`	Express app. `GET /healthz` for probes. `POST /mcp` : check the bearer token, make a new `McpServer` + `StreamableHTTPServerTransport` (stateless), handle the request.
`Dockerfile`	`node:24-alpine` , `npm ci --omit=dev` , runs as user 1000 (not root).

6.3 The tools

Tool	Inputs	What it calls
`list_products`	search?, limit?	GET /wc/v3/products
`create_product`	name, price, description?	POST /wc/v3/products (simple, published)
`list_orders`	date? ("today" or YYYY-MM-DD), status?, limit?	GET /wc/v3/orders with after/before for that whole day
`get_order`	id	GET /wc/v3/orders/{id}
`update_order_status`	id, status (enum)	PUT /wc/v3/orders/{id}
`ping`	none	No store call. Same name exists on the other servers, to show collision handling.

"Today": the server knows the store time zone (`STORE_TIMEZONE` , default Asia/Kolkata, also set in WordPress). It computes today's date in that zone and asks for orders between 00:00:00 and 23:59:59. So "what orders came in today" matches what the shop owner sees.

6.4 Stateless Streamable HTTP (the core code)

```
app.post("/mcp", checkToken, async (req, res) => {
  const server = new McpServer({ name: "woocommerce", version: "1.0.0" });
  addTools(server);
  const transport = new StreamableHTTPServerTransport({
    sessionIdGenerator: undefined,  // stateless: no sessions
    enableJsonResponse: true,       // plain JSON answers
  });
  res.on("close", () => { transport.close(); server.close(); });
  await server.connect(transport);
  await transport.handleRequest(req, res, req.body);
});
```

Why stateless? Each request stands alone, so there is nothing to keep in memory, any pod can answer, and the code is short. GET and DELETE on `/mcp` answer 405 (they are only for sessions/streams).

7. Part 2: the two extra MCP servers

The task asks for at least three upstream servers. We wrote two small ones in the same style, because they are reliable, run inside the cluster (so we can kill their pods in the demo), and each has its own token.

Server	Tools	Why it is useful in the demo
`notes-mcp-server`	add_note, list_notes, delete_note, ping	Two-server prompt: "list today's orders and save a summary as a team note".
`utils-mcp-server`	get_current_time, convert_currency, slow_task, ping	"Total of today's orders in EUR" (two servers). `slow_task` shows the gateway timeout.

Notes are kept in memory (a list), so they are gone after a restart. That is fine for a demo server.

8. Part 3: the gateway (file by file)

Folder: `gateway/src/`. It is two things at once: an **MCP client** to every upstream server, and an **MCP server** at `/mcp` for everyone else. It also serves the dashboard API.

File	What it does
`config.js`	All settings from environment variables: port, data folder, encryption key, first admin, timeouts (connect 5 s, call 20 s, health check every 15 s), LLM settings, seed servers.
`db.js`	Opens `/data/gateway.db` and makes the tables: users, sessions, api_keys, servers, audit_log, settings. Adds the `admin_only_tools` column for older databases.
`security.js`	Passwords: scrypt + salt. API keys and login tokens: random, stored as SHA-256 hash. Upstream credentials: AES-256-GCM encrypt/decrypt with the key from the Secret.
`upstreams.js`	The MCP client side. Builds headers with the decrypted credential, opens a client (`StreamableHTTPClientTransport`) with a timeout, health-checks all servers in parallel, keeps status + tools in memory, calls tools, turns errors into clear text. Also the tool permission helpers.
`auth.js`	Reads `Authorization: Bearer ...` . Login tokens (`sess_`) for the dashboard and the chat agent. API keys (`mcpgw_`) for MCP clients. Revoked key -> 401 and an audit row for that user.
`audit.js`	Writes and reads audit rows: user, via (chat / mcp-client), server, tool, start time, duration, ok, error.
`mcp-endpoint.js`	The gateway's MCP server (low-level `Server` class). `tools/list` returns renamed tools. `tools/call` splits the name, runs checks, calls the upstream, writes the audit row.
`url-check.js`	SSRF protection for server URLs (see section 13).
`routes/login.js`	POST /api/login, POST /api/logout, GET /api/me. Login tokens last 12 hours.
`routes/servers.js`	List / add / edit / disable / remove / check servers. Admin-only for changes. Never returns secrets.
`routes/users.js`	Admin invites a member (makes a one-time password), disables/enables users.
`routes/keys.js`	Make (shown once), list (prefix only) and revoke API keys. Members: own keys. Admins: all.
`routes/audit.js`	GET /api/audit (admin: all, member: own), GET /api/tools.
`chat.js` , `routes/chat.js`	The chat agent (section 9).
`seed.js`	First start only: create the first admin from the Secret, register WooCommerce, notes and utils servers (tokens encrypted). A flag makes sure removed servers do not come back.
`index.js`	Wires it all together: `/healthz` , `/mcp` , `/api/*` , starts health checks.

8.1 Tool aggregation

Every 15 seconds the gateway connects to every enabled server **at the same time** (`Promise.allSettled`), lists its tools, and saves the result. `tools/list` on the gateway reads this saved list, so it is fast and never waits for a slow server.

8.2 Name collisions

> **Our scheme:** every tool becomes `<server-name>__<tool-name>`. Server names may only use a-z, 0-9 and "-" (no "_"), so splitting at the first "__" is always right. Example: `woocommerce__ping`, `notes__ping`, `utils__ping` are three different tools.

8.3 Timeouts and dead servers

- Connect + list: 5 seconds. Tool call: 20 seconds. After that the caller gets: `Call to 'utils' failed: Timeout: the server did not answer in time`.
- A dead server becomes **Unreachable** with the reason (for example "Connection refused" or "Timeout"). Its tools leave the list. Other servers are not affected, because every check and every call is separate.
- Disabled server: tools leave the list, and direct calls get "Server 'x' is disabled".

8.4 Tool permissions (stand-out item 2)

An admin clicks a tool in the dashboard to lock it ("admin only"). Locked tool names are stored per server. For members, locked tools are **hidden** from `tools/list` and **rejected** if called directly ("You are not allowed to use ..."), and the rejection is in the audit log.

9. Part 4: the chat agent and its tool loop

File: `gateway/src/chat.js`.

```
Browser --POST /api/chat (login token, messages)--> gateway
  1. MCP client --> http://127.0.0.1:4000/mcp  with  Authorization: Bearer <the user's login
token>
     (so the agent IS the logged-in user: same tools, same permissions, same audit log)
  2. tools = client.listTools()  --> turn into LLM "function" tools
  3. LOOP (max 8 rounds):
       answer = LLM(messages, tools)
       if answer has no tool calls  -> return answer + list of tool calls
       for each tool call:
           result = client.callTool(name, args)   (goes through the gateway: checks + audit)
           add result to messages as role "tool"
  4. The UI shows the answer and, under it, every tool that was called (name, inputs, result).
```

- **Why call our own /mcp instead of calling functions directly?** Then the agent follows exactly the same path as Claude Code: same auth, same permissions, same audit. Nothing talks to the upstream servers except the gateway.
- **LLM:** any OpenAI-style "chat completions" API. Default Groq (free). The key is in a Secret (`llm-secret`) and only the gateway pod has it.
- **Safety:** the system prompt says tool results are data, not orders. Each tool result sent to the model is cut to 8 KB. Max 8 rounds.

10. Part 5: the React dashboard

Page	Who	What
Login	all	Email + password, gets a login token (stored in the browser).
MCP Servers	all (changes: admin)	Status badge (Connected / Unreachable / Disabled), reason, last check, tools. Admin: add, edit (URL, credential), disable/enable, remove, click a tool to lock it. Refreshes every 5 s.
Chat	all	Chat with the agent. Shows the tools called, with inputs and results.
API Keys	all	Generate my key (shown once, with the ready Claude Code command and Cursor config), list, revoke.
Members	admin	Invite (one-time password), issue a key for a user, disable/enable.
Audit Log	all (member: own rows)	Time, user, via, server, tool, duration, success/failed + error.

The dashboard only knows the gateway's `/api`. It never sees an upstream credential: the API returns only `hasCredential: true/false`.

11. Part 6: Kubernetes and Helm (template by template)

Chart: `helm/team-mcp-gateway/`. Everything goes into one namespace, `mcp`.

Template	Objects	Notes
`secrets.yaml`	Secret `app-secrets`	Random passwords/tokens made by Helm (`randAlphaNum`). On upgrade, `lookup` reads the old Secret and keeps the same values.
`mariadb.yaml`	Service, StatefulSet + volumeClaimTemplate	User 999. Readiness: `healthcheck.sh --connect --innodb_initialized`. Liveness: TCP 3306.
`wordpress.yaml`	PVC `wordpress-files`, Service, Deployment (Recreate)	User 33 (www-data). Safe sysctl `ip_unprivileged_port_start=0` lets it open port 80 without root. Readiness: /wp-login.php. Liveness: a static file.
`store-setup.yaml`	ConfigMap (scripts), Job (hook)	post-install + post-upgrade. WP-CLI: install WordPress, WooCommerce, settings, Cash on delivery, REST key, demo products. Every step is safe to repeat.
`woo-mcp-server.yaml`	Service (ClusterIP), Deployment	No Ingress. Probes on /healthz.
`notes-/utils-mcp-server.yaml`	Service, Deployment	Same.
`gateway.yaml`	PVC `gateway-data`, Service, Deployment (Recreate)	Gets the encryption key, first admin password, upstream tokens and the optional LLM key from Secrets.
`dashboard.yaml`	Service, Deployment	nginx on 8080, writable /tmp only.
`ingress.yaml`	2 Ingress	store host -> WordPress. dashboard host: /api and /mcp -> gateway, / -> dashboard. Optional TLS with cert-manager.
`networkpolicies.yaml`	7 NetworkPolicies	Deny all incoming, then allow only what is needed (section 13).

11.1 Probes, requests and limits

Every workload has a readiness and a liveness probe, and CPU/memory requests and limits (set in `values.yaml`). Example: WordPress requests 100m CPU / 256Mi and may use up to 1 CPU / 768Mi. The small MCP servers request 20m / 48Mi.

11.2 Storage

Three volumes: MariaDB data, WordPress files (plugins, uploads), gateway SQLite file. They survive pod restarts and Helm upgrades. The storage class is a value: Kind's default locally, `local-path` on k3s.

11.3 Local domains

We use `localtest.me`: public DNS where every name under it points to 127.0.0.1. Kind maps ports 80/443 of the laptop to the ingress controller. So `http://store.localtest.me` and `http://dashboard.localtest.me` work with no hosts file changes.

12. How one tool call travels, step by step

Example: user B in Claude Code asks "what orders came in today?"

1. Claude Code sends `POST http://dashboard.localtest.me/mcp` with `Authorization: Bearer mcpgw_...` and the JSON-RPC message `tools/call`, name `woocommerce__list_orders`, arguments `{"date":"today"}`.
2. The Ingress sends `/mcp` to the gateway Service (allowed by NetworkPolicy: from the ingress namespace).
3. `requireMcpAuth` hashes the key, finds it, checks it is not revoked and the user is active. The request now knows "this is user B".
4. The gateway's MCP server splits the name: server `woocommerce`, tool `list_orders`. Checks: server exists, enabled, tool exists, B's role may use it.
5. `callUpstreamTool` decrypts the WooCommerce MCP token, opens an MCP client to `http://woo-mcp-server:3000/mcp` (allowed: only from the gateway), runs `initialize` and `tools/call` with a 20 s timeout.
6. The WooCommerce MCP server checks the token, computes today's date in the store time zone, and calls `GET http://wordpress/?rest_route=/wc/v3/orders&after=...&before=...` with the consumer key/secret (allowed: woo-mcp -> wordpress).
7. WordPress reads MariaDB (allowed: wordpress -> mariadb) and returns the orders.
8. The answer goes back the same way. The gateway writes the audit row (user B, mcp-client, woocommerce, list_orders, time, duration, ok) and returns the result to Claude Code.

13. Security: keys, secrets, isolation

13.1 Where secrets live

Secret	Stored as
DB passwords, WP admin password, WooCommerce key/secret, MCP tokens, encryption key, first admin password	Kubernetes Secret `app-secrets` (random, made at install)
LLM API key	Kubernetes Secret `llm-secret` (you create it)
Upstream credentials inside the gateway	AES-256-GCM encrypted in SQLite
User API keys and login tokens	SHA-256 hash only (key shown once)
Dashboard passwords	scrypt hash with salt

No password is in Git or in a values file. The browser and MCP clients never get an upstream credential.

13.2 What a member can and cannot do

- **Can:** log in, generate and revoke their own API keys, use all allowed tools (dashboard chat or external client), see their own audit rows, see the server list (without secrets).
- **Cannot:** add/edit/remove servers, invite or disable users, see other people's keys or audit rows, use locked (admin-only) tools, see any upstream credential, reach any MCP server directly.

13.3 Public vs internal (and how it is enforced)

- **Public:** only the two Ingress hosts (store, dashboard with /api and /mcp).
- **Internal only:** MariaDB, the three MCP servers (ClusterIP, no Ingress).
- **NetworkPolicies** (enforced by the cluster network): deny all incoming; allow ingress controller -> WordPress/dashboard/gateway; WordPress + setup Job -> MariaDB; woo-mcp -> WordPress; **gateway only** -> MCP servers. Egress: woo-mcp may only reach WordPress + DNS; notes/utils only DNS.
- **Tested:** a random pod in the namespace could not connect to `woo-mcp-server:3000`; the gateway could.
- **Containers:** all non-root. Our images also: read-only root filesystem, all Linux capabilities dropped, no privilege escalation.

13.4 If an admin adds a malicious MCP server

- Its tools are namespaced (`evil__...`), so it cannot replace `woocommerce__*` tools.

- It only gets its own credential, never user keys (we do not pass client tokens through).
- Descriptions are labeled with the server name and cut to 1000 characters. The chat agent is told tool results are data. (Prompt injection is still a real risk; models can be fooled.)
- Timeouts stop it from hanging the gateway. The SSRF check stops URLs that point to localhost or cloud metadata (169.254.169.254).
- The admin can disable or remove it in one click, and the audit log shows every call it got.

Full details: `docs/threat-model.md`.

14. Problems we hit and how we fixed them

Problem	Why it happened	Fix
`kind load docker-image` failed ("content digest not found")	Docker Desktop's multi-platform image store; Kind tries to import all platforms.	`scripts/build-images.sh` saves only `linux/amd64` and imports with `ctr` into the Kind node.
Pods stuck in `CreateContainerConfigError`	`runAsNonRoot: true` but the image said `USER node` (a name). Kubernetes needs a number to check.	`USER 1000` in the Dockerfiles (same user, as a number).
Setup Job failed: "WooCommerce needs WordPress 7.0"	The newest WooCommerce (11.2) needs WordPress 7; we started with the 6.8 image.	Use `wordpress:7.0-php8.3-apache`; clean reinstall.
WordPress as non-root cannot open port 80	Ports below 1024 need root.	Safe sysctl `net.ipv4.ip_unprivileged_port_start=0` on the pod. (Editing Apache's config was not possible as non-root.)
Product links showed `https://` locally	Our `X-Forwarded-Proto: https` header makes WordPress build https links.	The MCP server rewrites links to `STORE_PUBLIC_URL`.

15. Testing: what we checked and what we saw

We used `scripts/smoke-test.sh` (curl through the real Ingress) plus some manual checks.

Check	Result
Storefront and dashboard through Ingress	HTTP 200 for both
Checkout with Cash on delivery (Store API, same as the browser checkout)	Order #14 created, status "processing"
All three servers registered and Connected	woocommerce (6 tools), notes (4), utils (4)
User B (member, own key) lists tools	14 tools, all with server prefix
User B: today's orders	Returned order #14 with items, total, customer
Three tools named `ping`	Each routed to the right server
Slow tool (25 s)	Stopped at 20 s with "Timeout: the server did not answer in time"; audit row failed, 20008 ms
Revoke B's key	Next call: 401 "API key was revoked"; audit row under user B
Disable notes server	0 notes tools left; enable again -> Connected
Kill an upstream (scale notes to 0)	notes Unreachable with reason; WooCommerce and utils still work
Tool permissions	Locked create_product hidden from member and rejected if called; admin sees all 6
NetworkPolicy	Random pod cannot reach woo-mcp-server; gateway can
Helm upgrade and rollback	Rollback to revision 1 and upgrade again worked; same Secrets and data
Chat agent	Needs your LLM key in the `llm-secret` Secret. Without a key it answers "no LLM API key". (Run the chat test after adding the key.)

16. How to run it

16.1 Local (Kind)

```
kind create cluster --config scripts/kind-config.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-
v1.12.1/deploy/static/provider/kind/deploy.yaml
sh scripts/build-images.sh

kubectl create namespace mcp
kubectl -n mcp create secret generic llm-secret --from-literal=LLM_API_KEY=<groq-key>

helm install tmg helm/team-mcp-gateway -n mcp --create-namespace \
  -f helm/team-mcp-gateway/values-local.yaml --wait --timeout 15m

# passwords
kubectl -n mcp get secret app-secrets -o jsonpath="{.data.GATEWAY_ADMIN_PASSWORD}" | base64 -d
kubectl -n mcp get secret app-secrets -o jsonpath="{.data.WP_ADMIN_PASSWORD}" | base64 -d

bash scripts/smoke-test.sh
```

Open `http://store.localtest.me` and `http://dashboard.localtest.me`.

16.2 Connect Claude Code

```
claude mcp add --transport http team-gateway http://dashboard.localtest.me/mcp \
  --header "Authorization: Bearer <your-api-key>"
```

16.3 Production (VPS with k3s)

Same chart, `values-prod.yaml`. Changes: image registry (ghcr.io), Traefik instead of nginx, real domains, TLS with cert-manager, `local-path` storage, bigger volumes, ingress controller namespace `kube-system`. Steps: install k3s, cert-manager, ClusterIssuer, push a git tag to build images, create `llm-secret`, `helm install ... -f values-prod.yaml`. Full steps: `docs/production-vps.md`.

16.4 Upgrade and rollback

```
helm upgrade tmg helm/team-mcp-gateway -n mcp -f helm/team-mcp-gateway/values-prod.yaml --set
images.tag=1.0.1 --wait
helm -n mcp history tmg
helm -n mcp rollback tmg <revision> --wait
```

> Helm rolls back Kubernetes objects, **not data**. Take a database backup before risky upgrades (commands in `docs/production-vps.md`).

17. Demo video plan (about 12-15 minutes)

1. **Intro (1 min):** the 4 layers, the architecture picture.
2. **Kubernetes tour (2 min):** `kubectl -n mcp get pods,svc,pvc,ingress,networkpolicy,secret`. Show probes and limits in `values.yaml`. Point out: only 2 Ingress hosts, MCP servers are ClusterIP.
3. **Store (1 min):** buy a product with Cash on delivery; show it in WooCommerce -> Orders.
4. **Admin dashboard (1 min):** servers with status and tools; invite user A and user B.
5. **User A chat (2 min):** "Create a product called Demo Lamp with price 30" -> show it in the store. Buy it by hand. "What orders came in today?" Show the tool calls under the answer.
6. **User B external client (2 min):** make a key, `claude mcp add ...`, list tools, ask the orders question, then "List today's orders and save a summary as a team note" (two servers).
7. **Control (2 min):** revoke B's key -> rejected. Disable notes -> tools gone (chat + reconnect the client). `kubectl -n mcp delete pod -l app=utils-mcp-server` or scale to 0 -> Unreachable, WooCommerce still works.
8. **Audit log (30 s):** every call with the right user.
9. **MCP internals + security (2 min):** code walk of `upstreams.js` and `mcp-endpoint.js`; name scheme, timeouts; hashing and encryption; malicious server answer.
10. **Production story (1 min):** diff of values-local vs values-prod; upgrade and rollback.

18. Interview questions and simple answers

Q: Why is the gateway both an MCP client and an MCP server?
A: To the outside it is one MCP server (one URL, one key). To the upstreams it is a normal MCP client. It joins the tool lists and routes each call.

Q: How do you handle two servers with the same tool name?
A: Every tool gets the server name in front: `server__tool`. Server names cannot contain "_", so the split is never wrong. We show it with three `ping` tools.

Q: What happens if an upstream is slow or dead?
A: Listing uses a cache that is refreshed in the background in parallel, so it never waits. Calls have a 20 s timeout and return a clear error. A dead server is marked Unreachable with the reason; others keep working.

Q: How are API keys stored?
A: Only the SHA-256 hash and a short prefix. The full key is shown once. On each request we hash the given key and look it up. Revoked keys fail at once.

Q: Why SHA-256 for keys but scrypt for passwords?
A: Keys are 48 random hex characters, so they cannot be guessed; a fast hash is fine. Passwords are chosen by people and can be guessed, so we use a slow hash with salt.

Q: Why encrypt upstream credentials instead of hashing them?
A: We must send them to the upstream, so we need the real value back. Hashing is one-way; encryption is two-way. The key is in a Kubernetes Secret, not in the database.

Q: Why don't you pass the client's token to the upstream?
A: The upstream could steal and replay it against our gateway (confused deputy). Tokens should only be accepted by the service they were made for. The gateway checks the user, then uses the upstream's own credential.

Q: How does the chat agent act as the logged-in user?
A: It connects to the gateway's own `/mcp` with the user's login token. So it gets that user's tools and permissions, and every call is in the audit log under that user with via = chat.

Q: Why stateless Streamable HTTP?
A: Simple code, no session memory, any pod can answer. The cost: no server push, so clients must re-list tools to see a change.

Q: Where is isolation enforced?
A: Three places: (1) no Ingress for MCP servers and the DB (ClusterIP only); (2) NetworkPolicies, deny by default, MCP servers accept only the gateway; (3) a bearer token on each MCP server, known only to the gateway.

Q: How do secrets survive a Helm upgrade?
A: The Secret template uses `lookup` to read the existing Secret. If a value is there, it is reused; if not, a new random value is made.

Q: What would break with two gateway replicas?
A: The MCP endpoint is stateless and would be fine. SQLite on a ReadWriteOnce volume is the problem. We would move to Postgres. The health cache is per pod, which is fine.

Q: How does the WooCommerce MCP server authenticate to the store?
A: A WooCommerce REST API key (consumer key + secret). The setup Job writes it into WooCommerce's key table (hashed, like WooCommerce does), using values from the Secret. The MCP server gets the same values as env vars.

Q: Is the setup Job safe to run twice?
A: Yes. It checks "is WordPress installed?", "is the plugin installed?", "are there products?", and it replaces the API key row instead of adding a second one.

19. Limits and next steps

- **One gateway replica** (SQLite). Next: Postgres + 2 replicas + rate limits per key.
- **New upstream connection per call.** Next: a small connection pool.
- **No push of tool list changes** to clients (stateless). Clients must reconnect to see changes.
- **Prompt injection** from untrusted servers is reduced, not solved. Next: "pending review" for new servers, per-server tool allow-lists.
- **SSRF check** does not cover DNS rebinding.

- **Invites** are one-time passwords, not email links. **OAuth** on the gateway is not done.
- **Bonus multi-store provisioning** was not attempted; we focused on a working core.

End of guide.